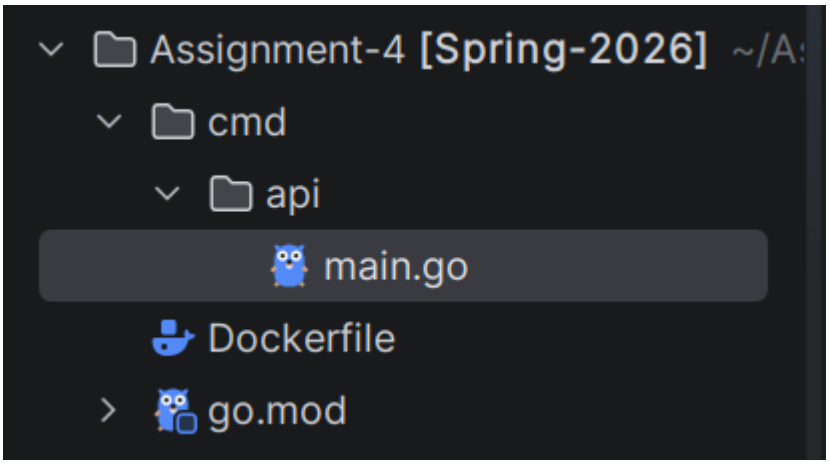
Before starting, make sure you've read the following articles:

- Writing a Dockerfile
- Docker Compose Quickstart

Step 1: Write or found written `go` project.

Below, is the example project. You can use your own projects from the `previous` lessons:




```go
105 func main() {
106
107     r := mux.NewRouter()
108
109     movies = append(movies, Movie{ID: "1", Genre: "horror", Budget: 500000, Title: "SAW", Technicians: Tec
110
111     movies = append(movies, Movie{ID: "2", Genre: "Romance", Budget: 1000000, Title: "TEST", Technicians:
112
113     fmt.Println(movies)
114
115     r.HandleFunc("/movies", getmovies).Methods(methods...: "GET")
116     r.HandleFunc("/movies/{id}", getmovie).Methods(methods...: "GET")
117     r.HandleFunc("/movies", createMovies).Methods(methods...: "POST")
118     r.HandleFunc("/movies/{id}", updateMovies).Methods(methods...: "PUT")
119     r.HandleFunc("/movie/{id}", deletemovie).Methods(methods...: "DELETE")
120     fmt.Println(a...: "Starting the Server...")
121     log.Fatal(http.ListenAndServe(addr: ":8000", r))
122 }
```

Step 2: Create `Dockerfile` file.

> In this step, we will build our project using `single-stage` build and `multi-stage` build approaches and notice the practical difference.

Step 2.1 Let's start with the single-stage:

Create and open `Dockerfile`:

```
FROM golang:1.25.5-alpine

WORKDIR /app

COPY . .

RUN go build -o myapp ./cmd/api/main.go
CMD [ "./myapp" ]
```

> This image contains the entire `Go` toolchain, although it is not needed for production.

> Let's build it! Use the command below:

```
docker build -t my-go-app .
```

> It will take some time to complete and you will get something like this:

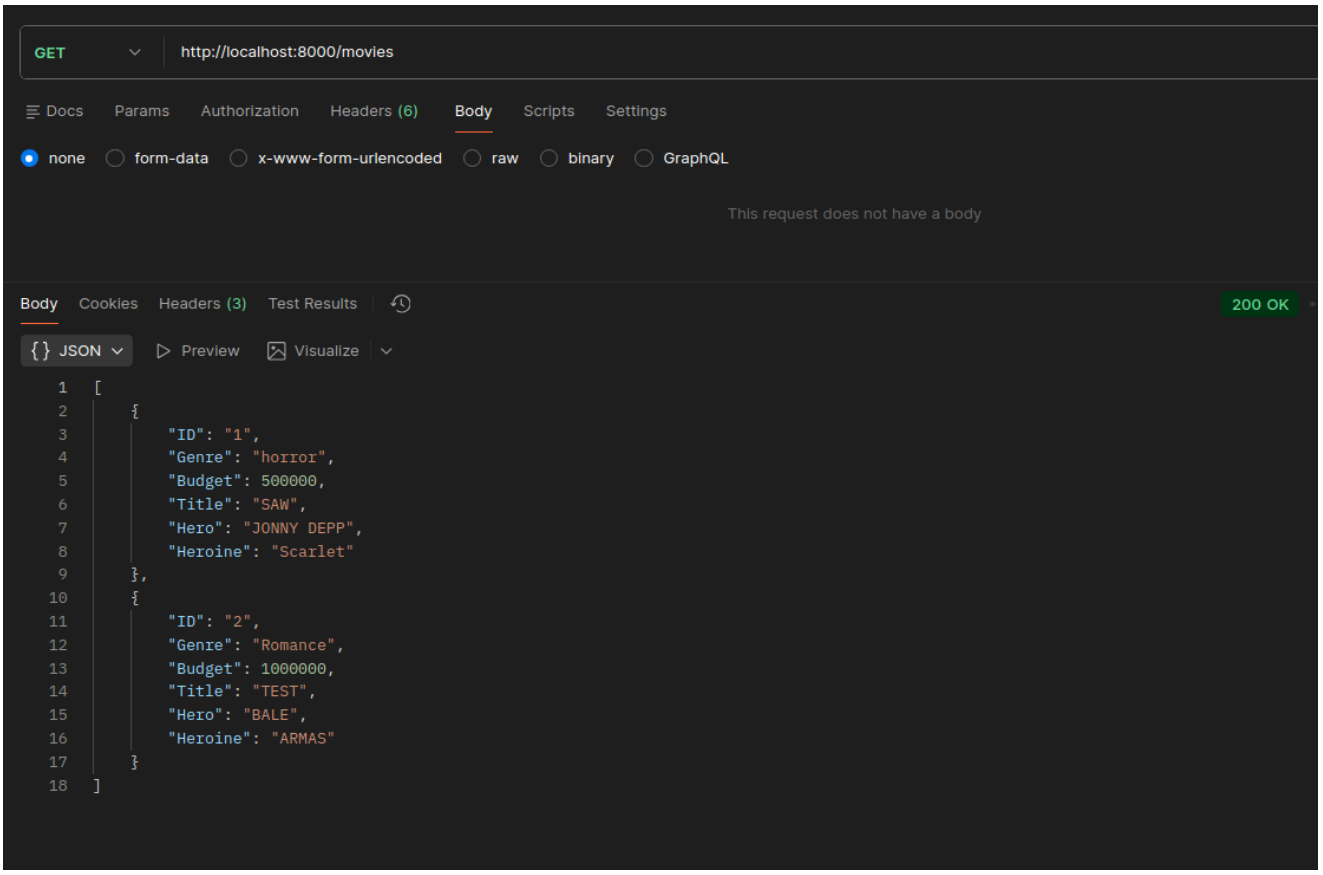
```
bakytzhan@Bakytzhan:~/Assignment-4$ docker build -t my-go-app .
[+] Building 10.0s (12/12) FINISHED                                                                                docker:default
 => [internal] load build definition from Dockerfile                                                                          0.0s
 => => transferring dockerfile: 209B                                                                                          0.0s
 => [internal] load metadata for docker.io/library/alpine:latest                                                              0.4s
 => [internal] load metadata for docker.io/library/golang:1.25.5-alpine                                                       0.4s
 => [internal] load .dockerignore                                                                                             0.0s
 => => transferring context: 2B                                                                                               0.0s
 => [builder 1/4] FROM docker.io/library/golang:1.25.5-alpine@sha256:ac09a5f469f307e5da71e766b0bd59c9c49ea460a528cc3e6686513d64a6f1fb  0.0s
 => [stage-1 1/2] FROM docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659      0.0s
 => [internal] load build context                                                                                             0.0s
 => => transferring context: 415B                                                                                             0.0s
 => CACHED [builder 2/4] WORKDIR /app                                                                                         0.0s
 => [builder 3/4] COPY . .                                                                                                    0.1s
 => [builder 4/4] RUN go build -o myapp ./cmd/api/main.go                                                                     9.3s
 => CACHED [stage-1 2/2] COPY --from=builder app/myapp /myapp                                                                 0.0s
 => exporting to image                                                                                                        0.0s
 => => exporting layers                                                                                                       0.0s
 => => writing image sha256:57cb699d062f80ecddc9e9fa683466f53bbb9918d24dc2f1f01e0200ce95ef63                                  0.0s
 => => naming to docker.io/library/my-go-app                                                                                  0.0s
```

> Now, we will run our docker image we've built:

```
docker run -p 8000:8000 my-go-app
```

```
^Cbakytzhan@Bakytzhan:~/Assignment-4$ docker run -p 8000:8000 my-go-app
[{1 horror 500000 SAW {JONNY DEPP Scarlet}} {2 Romance 1000000 TEST {BALE ARMAS}}]
Starting the Server...
SS
```

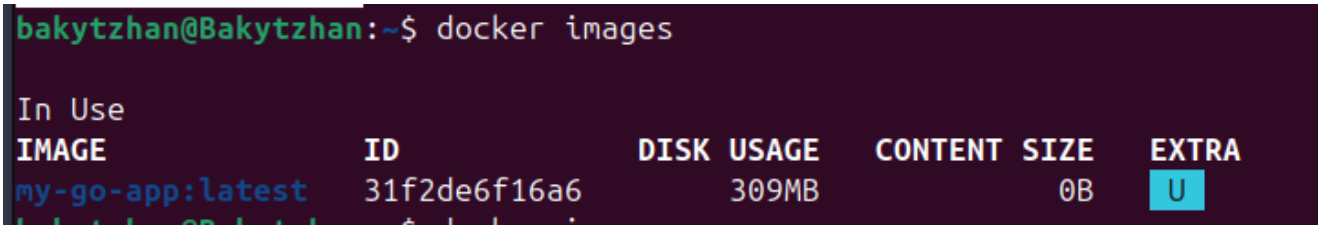
> As you can see, the app is running and is ready to serve the http requests. Let's check!



> **As we've built our docker image, let's see the information about it using docker commands:**

> **Run:**

```
docker images
```



> **Our image takes `309MB` of `DISK USAGE`. That's quite a lot. Just remember this number for now.**

Step 2.2 Multi-stage build

> **In the Dockerfile make some changes:**

```
FROM golang:1.25.5-alpine AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp ./cmd/api/main.go

FROM alpine
```

```
COPY --from=builder app/myapp /myapp
ENTRYPOINT ["/myapp"]
```

> Repeat the `build` process:

```
docker build -t my-go-app .
```

Let's check the `docker images` again:

```
docker images
```

```
bakytzhan@Bakytzhan:~$ docker images

IMAGE              ID             DISK USAGE   CONTENT SIZE   EXTRA
my-go-app:latest   57cb699d062f       17.3MB             0B
bakytzhan@Bakytzhan:~$
```

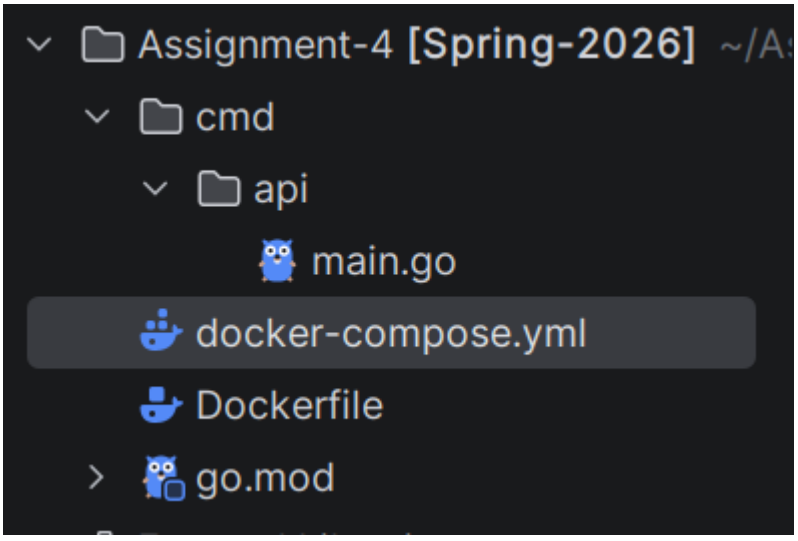
> **Result: The final image weighs only a few megabytes and contains only the executable file.**

> **Since this image is very small, it easier for portability and can be easily to deploy in production. Although the multi-stage build sounds like a fantastic idea, there are certain scenarios when this should be used and certain scenarios when this should be avoided.**

> **Read more:**
>
> [Optimizing Golang Docker images with multi-stage build]
> [Что такое Multi-Stage Builds в Docker и зачем они нужны]

Step 3: Define a `docker-compose.yml`

> For this part, create `docker-compose.yml` file



> In the `docker-compose.yml`:

```yaml
services:
  web-app:
    container_name: web-app
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
```

> Now, let't build, create, and start all our service we've defined

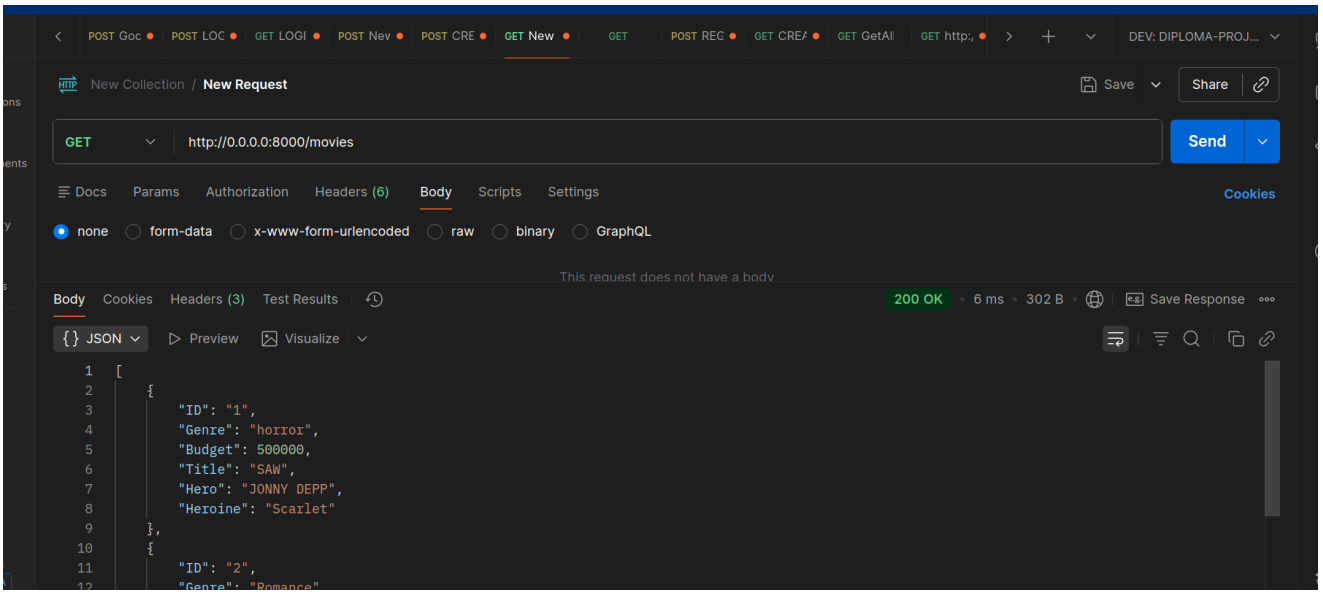
```
docker compose up --build -d
```

> Your pc may not recognize `docker compose`, if some error occurs, try `docker-compose` instead.

```
bakytzhan@Bakytzhan:~/Assignment-4$ docker compose up --build -d
[+] Building 1.8s (16/16) FINISHED
 => [internal] load local bake definitions                                                                    0.0s
 => => reading from stdin 522B                                                                                0.0s
 => [internal] load build definition from Dockerfile                                                          0.0s
 => => transferring dockerfile: 210B                                                                          0.0s
 => [internal] load metadata for docker.io/library/alpine:latest                                              1.4s
 => [internal] load metadata for docker.io/library/golang:1.25.5-alpine                                       1.6s
 => [auth] library/alpine:pull token for registry-1.docker.io                                                 0.0s
 => [auth] library/golang:pull token for registry-1.docker.io                                                 0.0s
 => [internal] load .dockerignore                                                                             0.0s
 => => transferring context: 2B                                                                               0.0s
 => [builder 1/4] FROM docker.io/library/golang:1.25.5-alpine@sha256:ac09a5f469f307e5da71e766b0bd59c9c49ea460a528cc3e6686513d64a6f1fb   0.0s
 => [internal] load build context                                                                             0.0s
 => => transferring context: 272B                                                                             0.0s
 => [stage-1 1/2] FROM docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659   0.0s
 => CACHED [builder 2/4] WORKDIR /app                                                                         0.0s
 => CACHED [builder 3/4] COPY . .                                                                             0.0s
 => CACHED [builder 4/4] RUN go build -o myapp ./cmd/api/main.go                                              0.0s
 => CACHED [stage-1 2/2] COPY --from=builder /app/myapp /myapp                                                0.0s
 => exporting to image                                                                                        0.0s
 => => exporting layers                                                                                       0.0s
 => => writing image sha256:4ee42cf8e7262c6ab5895b988b596ef39e64709258f37a288a8376a557574ae7                  0.0s
 => => naming to docker.io/library/assignment-4-web-app                                                       0.0s
 => resolving provenance for metadata file                                                                    0.0s
[+] up 3/3
 ✔ Image assignment-4-web-app   Built                                                                         1.9s
 ✔ Network assignment-4_default Created                                                                       0.0s
 ✔ Container web-app            Created                                                                       0.1s
```

> Wonderful!

Let's see whether we can access our project:



> Super!
>
>

> **Individual Assignment.** `3 points`

Technical Requirements:

1. Database Orchestration

Modify your `docker-compose.yml` to include a new service named `db`.

- Image: Use `postgres:15-alpine` or any official image of your choice (e.g., MySQL, MongoDB, Postgres).
- Credentials: Use Environment Variables (e.g., `POSTGRES_PASSWORD`) to set the `username`, `password` , and `database name`.
- Persistence: Create a Named Volume and mount it to `/var/lib/postgresql/data`. This ensures your database entities aren't deleted if the container is removed.
- Network Health: Implement a `healthcheck` using `pg_isready` to ensure the database is fully "Ready" before other services try to connect.
- Schema Initialization: Provide an `init.sql` file(or whole migration dir) mapped to `/docker-entrypoint-initdb.d/` in the `db` service to

automatically create your tables on startup.

2. Backend Integration

Update your Go application to connect to the database container.

- Service Dependency: Update the `web-app` service to use `depends_on` with the `service_healthy` condition pointing to your `db` service.
- Connection String: Instead of hardcoding credentials in Go, pull the host, port, user, and password from environment variables defined in the `docker-compose.yml`.
- DB Connection: Your backend must connect to the database using the Docker Service Name (e.g., `db` or `postgres`) as the hostname, rather than `localhost` or `127.0.0.1`.
- Demonstrate HTTP `GET`, `POST`, `DELETE`, `UPDATE` requests handling using database data.

3. Demo video scenario

- Student runs `docker ps -a`. (Show that no containers for this specific project are currently running or existing.)
- Student runs `docker compose up --build`. The successful result is shown
- Student shows the logs where the Go app waits for the Database healthcheck before printing "Starting the Server."
- Demonstrate the database is running by showing its logs or using `docker exec` to list tables.
- Service processes the user requests from `Postman`
- Persistence Proof: After adding data to database, run `docker compose down`, then `docker compose up` again and show that the data is still there.

- Student runs `docker images`
  - `What to point out`: Highlight the size of your Go image. If you used a multi-stage build, point out how small it is (e.g., ~15-20MB) compared to the base Golang image (~400MB).

> ## Definition of Done:

- **The backend service is containerized using multi-stage build and runs on port `:8080`**
- **`docker-compose.yml` is written according to `Technical Requirements` without any errors and `docker compose up --build` builds the containerized application**
- **The backend connects to database using docker service name as the hostname**
- **Containerized application returns successful responses**
- **Demo video is recorded according to the scenario**

Deliverables:

- Push all the changes to your GitHub repository
- Submit the link to this repository(Please, submit the link as a **LINK ATTACHMENT**, not a file)
- Submit [1-2]-minute demo video of your project as a **VIDEO-FILE ATTACHMENT**. You can speed it up if it is too long.

Below are marking criteria:

Category	Criteria	Weight
Containerization	**Multi-stage Build Optimization:** The backend image is built using a multi-stage `Dockerfile`, resulting in a significantly reduced final image size (e.g., <30MB).	0.5 points
Orchestration	**Full Stack Integration:** `docker-compose up` launches both services. The backend successfully connects to the DB using the **Service Name** and environment variables.	0.5 points
Resilience	**Healthchecks & Dependencies:** The `web-app` service correctly uses `depends_on` with a `service_healthy` condition, ensuring no startup race conditions.	0.5 points
Persistence	**Stateful Storage:** A **Named Volume** is correctly implemented. Data persists after running `docker compose down` and restarting.	0.5 points
Security	**Environment Management:** No hardcoded credentials in Go code. Usage of an `.env` or `environment` section in Compose for all sensitive data.	0.5 points
Demonstration	**Video Scenario:** The video follows the required steps: showing empty `ps -a`, the build process, the log synchronization, and successful Postman testing.	0.5 points
OVERALL		`3 points`

> **Deadline: 01.03.2026, SUNDAY 23:59**

> **OPTIONAL FEATURES (Choose Freely)**

> Students may implement any number of the following to increase difficulty and engagement.

> 🐣 EASY: "The Baby Gopher"

- Custom Network Names: Instead of using the default Docker network, define a custom bridge network in your `docker-compose.yml` (e.g., `movie-net`) and assign both services to it.
- Aliases & Shortcuts: Create a simple `Makefile` or a bash script (`run.sh`) that contains the command `docker compose up --build`, so you can launch the whole project with just one word.
- Graceful Shutdown: Implement signal handling in your Go code (`os/signal`) so that when you stop the container, the Go app prints "Shutting down gracefully..." and closes the DB connection before exiting.

👨‍💻 MEDIUM: "The Gopher-at-Work"

- The "Scratch" Challenge: Instead of using `alpine` for your final stage, use `scratch` (the smallest possible empty image). This will require you to handle CA certificates and static linking perfectly.
- Postgres Migrations: Instead of a simple `init.sql`, use a migration tool (like `golang-migrate`) as a separate "init-container" that runs and finishes before your API starts.

🧙 ADVANCED: "The Gopher Wizard"

- The Nginx Reverse Proxy: Add a third container running Nginx. Configure it so that users cannot access the Go API directly on port 8000. Instead, they must go through Nginx on port 80, which "proxies" the request to the backend.
- Service Scaling: Configure your Compose file so that you can run `docker compose up --scale web-app=3`. This will require

adding a Load Balancer (like Nginx) to distribute traffic between the three identical Go containers.